Table des matières

Cours 11 — Contrôler la qualité 1
Ce que vous allez faire 1
Avant de commencer 2
Étape 1 — Un pipeline au vert qui ment · Tenir le pipeline 2
Étape 2 — Les règles de la qualité, `sql/controles.sql` · Tenir le pipeline 2
Étape 3 — Le juge, `controler.py` · Tenir le pipeline 4
Étape 4 — La troisième étape du pipeline · Tenir le pipeline 5
Étape 5 — Le pipeline qui juge · Tenir le pipeline 6
Étape 6 — Versionner · Tenir le pipeline 7
À retenir 7

Cours 11 — Contrôler la qualité

Cours 11/12 · La Brûlerie — Formation Data Engineering

Sources	**Ingestion**	**RAW**
d'où vient la donnée	aller la chercher, et la charger	la déposer, la ranger, la traiter
✓ ici · Tenir le pipeline, ce qui traverse les trois étapes : le code, l'orchestration, la qualité	✓	✓

Ce que vous allez faire

Depuis le cours 05, un pipeline qui échoue le dit, par son code de sortie. C'est la panne la moins grave. La plus grave passe au vert et ment : elle charge « rien de nouveau » quand il fallait dire « rien du tout ». Vous écrivez les règles de la qualité dans `sql/controles.sql`, chacune avec ses écarts et ce qu'elle déclenche, bloquer ou avertir ; un juge, `controler.py`, qui les passe toutes avant de décider ; puis le pipeline gagne une troisième étape, qui l'arrête sur une règle à bloquer.

`pipeline.py 2025-07-01`, au vert pour un jour jamais livré → quatre règles dans `sql/controles.sql` → `controler.py`, qui les juge toutes → la troisième étape du pipeline → le 30 juin averti, le 1er juillet bloqué

© 2026 iAlexMG. Tous droits réservés.

Le code complet est dans `Codes_11_DE.md`, à ouvrir après avoir essayé.

Avant de commencer

Une seule fenêtre de l'Anaconda Prompt sert à ce cours, placée dans `Projet_Brulerie`, pour le Bloc-notes, Python et Git. Le notebook n'en est pas.

```
conda activate formations
REM Attendu : (formations) au début de l'invite.
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
dir /b
REM Attendu : 01_de, plateforme et sources, plus ce qu'Analytics Engineering y a laissé.
```

Ce cours part de l'entrepôt chargé jusqu'au 30 juin, avec l'argent des lots du cours 08, `argent_lots`. S'il manque, reprenez l'étape 4 du cours 08.

Étape 1 — Un pipeline au vert qui ment · Tenir le pipeline

Le mois figé finit le 30 juin : la caisse n'a jamais exporté le 1er juillet. Demandez-le quand même au pipeline, comme le ferait l'orchestrateur la nuit du 2 juillet.

1.1 Lancer le pipeline pour le 1er juillet, puis lire son code de sortie.

```
python 01_de\pipeline.py 2025-07-01
REM Attendu : bronze : 1527 tickets rangés, puis raw : 75 fichiers au journal.
echo %ERRORLEVEL%
REM Attendu : 0.
```

Tout passe, code 0. Le chargeur dit vrai : il n'avait rien de nouveau à charger. Mais « rien de nouveau » et « rien du tout » s'affichent pareil, et un orchestrateur qui lit 0 passe au jour suivant comme si la caisse avait livré. Le chargeur ne sait pas ce qu'il devait recevoir ; il ne connaît que ce qu'il a reçu.

Le mois porte un autre défaut qui passe au vert, connu depuis le cours 01 : le 12 juin exporté deux fois. La `RAW` garde les deux exports, et c'est voulu ; qui additionne `raw_tickets` sans garder le dernier export d'un jour compte 58 tickets deux fois, sans la moindre erreur.

Étape 2 — Les règles de la qualité, `sql/controles.sql` · Tenir le pipeline

Une règle pose une question à laquelle une requête répond par un nombre, ses écarts : 0, et la règle est respectée. Chaque règle se range dans une famille que le métier appelle une dimension de la qualité, et chacune déclenche quelque chose quand elle est rompue :

Ce qu'elle déclenche	Ce qui se passe	Quand le choisir
bloquer	l'étape échoue, le code de sortie vaut 1, et rien ne se construit ensuite	quand tout ce qui suit serait faux

© 2026 iAlexMG. Tous droits réservés.

Ce qu'elle déclenche	Ce qui se passe	Quand le choisir
avertir	l'étape passe, et la règle rompue s'affiche, pour quelqu'un qui doit regarder	quand la suite sait traiter le défaut, mais qu'une personne doit le savoir

Quatre règles pour La Brûlerie :

Règle	Dimension	La question	Ce qu'elle déclenche, et pourquoi
1	fraîcheur	la caisse a-t-elle livré le jour demandé ?	bloquer : sans l'export du jour, tout ce qui se calcule sur ce jour est faux
2	volume	le bronze compte-t-il autant de tickets que la source, chacun une seule fois ?	bloquer : un ticket perdu ou compté deux fois fausse toute vente calculée dessus
3	unicité	la caisse a-t-elle exporté un jour plus d'une fois ?	avertir : la `RAW` garde les deux exports et le bronze ne prend que le dernier ; rien n'est faux, mais quelqu'un doit savoir que la caisse a recommencé un export
4	cohérence	l'argent dit-il, lot par lot, ce que le torréfacteur déclare ?	avertir : la caisse et la `RAW` restent justes ; un lot en écart se regarde, il n'arrête pas le café

La règle 2 est le contrôle de `bronze.sql`, que le cours 05 avait fait garde dans le code du bronze : une règle cachée dans une fonction. Elle rejoint les autres, au même endroit, avec ce qu'elle déclenche. La règle 4 est le témoin du cours 08, étape 5. Bloquer ne veut pas dire refuser la donnée : les règles passent après le chargement, et la `RAW` a déjà tout gardé. Ce qu'une règle à bloquer arrête, c'est ce qui se construirait dessus.

Le fichier rend une ligne par règle : son nom, ce qu'elle déclenche, ses écarts. Il tourne dans l'entrepôt, sur le journal des chargements du cours 03, dont la colonne `fichier` porte le nom de chaque fichier chargé, `tickets_2025-06-12_202506121542.csv` pour le premier export du 12 juin.

Pour remplir	L'outil
le jour demandé	`$jour`, un paramètre nommé, comme au cours 09 : `controler.py` lui donnera sa valeur
le jour d'un export, dans son nom	`split_part(fichier, '_', 2)` rend le deuxième morceau entre les _ : `2025-06-12` ; le premier, `tickets`, dit la livraison

© 2026 iAlexMG. Tous droits réservés.

Pour remplir	L'outil
un écart, ou aucun	`CASE WHEN … THEN 1 ELSE 0 END`
un écart dont le signe ne compte pas	`ABS(…)`
les tickets de la source, chacun une fois	`COUNT(DISTINCT ticket_id)`, comme le contrôle de `bronze.sql`
une règle sous l'autre	`UNION ALL`, vu au cours 06
le témoin du cours 08	ses quatre conditions, l'étape 5 du cours 08 : le premier craquement, la durée, la température de sortie et les lectures

2.1 Créer le fichier.

```
notepad 01_de\sql\controles.sql
```

Accepter la création, puis écrire le code à partir de ce pseudocode, les trous `____` compris :

```
-- 01_de/sql/controles.sql : les règles de la qualité, chacune avec ce qu'elle déclenche et ses écarts
-- 1. fraîcheur : la caisse a livré le jour demandé
SELECT 'la caisse a livré le jour' AS regle, 'bloquer' AS action,
       CASE WHEN COUNT(*) = 0 THEN 1 ELSE 0 END AS ecarts
FROM journal WHERE split_part(fichier, '_', 1) = 'tickets' AND split_part(fichier, '_', 2) = ____
-- 2. volume : le bronze compte autant de tickets que la source, chacun une seule fois
UNION ALL SELECT 'le bronze égale la source', 'bloquer',
       ABS((SELECT COUNT(*) FROM read_parquet('plateforme/lac/bronze/tickets/*/*.parquet'))
           - (SELECT ____ FROM read_csv('sources/caisse/*/tickets_*.csv')))
-- 3. unicité : un jour que la caisse a exporté plus d'une fois ; le bronze n'en garde que le dernier
UNION ALL SELECT 'un seul export par jour', '____', COUNT(*) - COUNT(DISTINCT ____)
FROM journal WHERE split_part(fichier, '_', 1) = 'tickets'
-- 4. cohérence : le témoin du cours 08, ce que les sondes ont vu contre ce que le torréfacteur déclare
UNION ALL SELECT 'argent égal au journal des lots', 'avertir', COUNT(*)
FROM argent_lots AS a JOIN raw_lots AS l USING (lot_id)
WHERE ____;
```

Enregistrer, puis fermer le Bloc-notes. Les noms des règles évitent l'apostrophe, qui devrait se doubler dans un texte SQL.

Étape 3 — Le juge, `controler.py` · Tenir le pipeline

Le juge lance les règles pour le jour demandé, les juge toutes, puis décide. Une exécution qui s'arrêterait à la première règle rompue cacherait les suivantes, et on corrigerait un défaut à la fois, une relance à la fois. Comme `charger.py`, le juge est un module à côté du pipeline, qui l'appelle : il prend le jour, rend en quelques mots ce qu'il a jugé, ou lève une exception, comme chaque étape depuis le cours 05.

© 2026 iAlexMG. Tous droits réservés.

Pour remplir	L'outil
donner sa valeur à `$jour`	`con.execute(texte, {"jour": …})`, le dictionnaire des paramètres nommés
une règle rompue	ses écarts dépassent 0
une règle à bloquer, rompue	`action == "bloquer" and ecarts > 0`
y en a-t-il au moins une ?	`any(condition for … in regles)` rend vrai dès qu'une ligne passe la condition

3.1 Créer le fichier.

```
notepad 01_de\controler.py
```

Accepter la création, puis écrire le code à partir de ce pseudocode, les trous ____ compris :

```
# 01_de/controler.py : juger les règles de la qualité pour un jour, et arrêter tout sur une règle à bloquer
import duckdb

def controler(jusqua):
    # 1. chaque règle et ses écarts, jugées dans l'entrepôt pour le jour donné
    texte = open("01_de/sql/controles.sql", encoding="utf-8").read()
    with duckdb.connect("plateforme/entrepot/brulerie.duckdb") as con:
        regles = con.execute(texte, {"jour": ____}).fetchall()
    # 2. une règle rompue se dit, avec ce qu'elle déclenche et ses écarts
    rompues = [f"{action} : {regle} ({ecarts})" for regle, action, ecarts in regles if ____]
    # 3. toutes jugées avant de décider : une seule règle à bloquer rompue arrête le pipeline
    if any(____ for regle, action, ecarts in regles):
        raise ValueError(", ".join(rompues))
    return ", ".join([f"{len(regles)} règles jugées"] + rompues)
```

Enregistrer, puis fermer le Bloc-notes.

Étape 4 — La troisième étape du pipeline · Tenir le pipeline

Trois retouches à `pipeline.py` : importer le juge ; retirer la garde du bronze, que la règle 2 reprend, et lui faire dire les deux nombres de son contrôle ; ajouter le juge au bout des étapes. Il passe après la `RAW`, puisque ses règles la lisent.

4.1 Ouvrir le pipeline.

```
notepad 01_de\pipeline.py
```


© 2026 iAlexMG. Tous droits réservés.

Sous `from charger import charger`, importer le juge :

```
from controler import controler
```

Remplacer la fonction `bronze` par celle-ci, le trou compris : elle rend les tickets du bronze, puis ceux de la source.

```
def bronze(jusqua):
    # 1. le bronze, refait en entier ; son contrôle se juge désormais avec les règles de la qualité
    source, tickets = duckdb.sql(open("01_de/sql/bronze.sql", encoding="utf-8").read()).fetchone()
    return ____
```

Dans `lancer`, ajouter le juge au bout des étapes :

```
    # 3. chaque étape tracée ; la première qui échoue arrête tout, et le code de sortie vaut 1
    for nom, etape in (("bronze", bronze), ("raw", raw), ("controles", ____)):
```

Enregistrer, puis fermer le Bloc-notes. Le fichier compte 29 lignes.

Étape 5 — Le pipeline qui juge · Tenir le pipeline

5.1 Relancer le mois.

```
python 01_de\pipeline.py 2025-06-30
REM Attendu : bronze : 1527 tickets rangés, 1527 à la source, puis raw : 75 fichiers au journal,
REM puis controles : 4 règles jugées, avertir : un seul export par jour (1).
echo %ERRORLEVEL%
REM Attendu : 0.
```

Entre parenthèses, les écarts de la règle rompue. La règle 3 avertit : la caisse a exporté un jour deux fois, le 12 juin, et le bronze n'en a gardé que le dernier export, 65 tickets. Le pipeline passe, code 0, et le dit. Elle avertira à chaque passage, puisqu'elle juge toute la `RAW`, qui garde les deux exports ; jugée sur le seul jour logique, elle ne sonnerait que le 13 juin.

5.2 Relancer le 1er juillet.

```
python 01_de\pipeline.py 2025-07-01
REM Attendu : bronze et raw comme en 5.1, puis controles : échec, bloquer : la caisse a livré le
REM jour (1), avertir : un seul export par jour (1).
echo %ERRORLEVEL%
REM Attendu : 1.
```

Le même chargement que l'étape 1, mais cette fois le pipeline sait ce qu'il devait recevoir : pas d'export du 1er juillet, la règle 1 bloque, code 1. Le juge a passé les quatre règles avant de décider : l'avertissement de la règle 3 s'affiche aussi.

© 2026 iAlexMG. Tous droits réservés.

Si `controles` échoue sur	La cause, et le geste
`ModuleNotFoundError: No module named 'controler'`, dès le lancement	`controler.py` n'est pas dans `01_de`, ou porte un autre nom
`Invalid Input Error: Values were not provided for the following prepared statement parameters: jour`	le dictionnaire des paramètres manque à `con.execute`, ou sa clé n'est pas `jour`
`Catalog Error: Table with name argent_lots does not exist!`	l'argent n'est pas dans l'entrepôt : le cours 08, étape 4.3, le refait

5.3 Rattraper du 29 juin au 2 juillet, comme l'orchestrateur après quelques nuits d'arrêt.

```
python 01_de\rattraper.py 2025-06-29 2025-07-02
REM Attendu : le 29 et le 30 juin passent, chacun avec son avertissement ; le 1er juillet bloque,
REM puis rattrapage arrêté au 2025-07-01 : les jours suivants attendent.
echo %ERRORLEVEL%
REM Attendu : 1.
```

Le rattrapage du cours 06 n'a pas changé : il s'arrête au premier jour en échec, et le premier jour en échec est maintenant celui que la caisse n'a pas livré. Le 2 juillet n'est pas tenté.

Étape 6 — Versionner · Tenir le pipeline

6.1 Valider les règles, le juge et le pipeline.

```
git -C 01_de add sql/controles.sql controler.py pipeline.py
git -C 01_de commit -m "Cours 11 : les règles de la qualité, jugées par le pipeline"
REM Attendu : 3 files changed.
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
```

À retenir

- Un pipeline qui ne plante pas peut mentir : « rien de nouveau » et « rien du tout » s'affichent pareil, tant qu'une règle ne dit pas ce qui devait arriver.
- Une règle compte ses écarts, et déclenche quelque chose quand elle est rompue : bloquer quand tout ce qui suit serait faux, avertir quand la suite sait faire mais qu'une personne doit savoir.
- Le juge passe toutes les règles avant de décider, après le chargement : bloquer arrête ce qui se construirait sur la donnée, sans la refuser.

© 2026 iAlexMG. Tous droits réservés.